

Intermediate (watch the model choose between two tools) — Solution

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Basic Version

Approach 1 — a plain counting loop

```
from langchain_core.tools import tool
from langchain_openai import ChatOpenAI

@tool
def get_weather(city: str) -> str:
    """Get the current weather for a city."""
    return f"Sunny in {city}"

@tool
def get_forecast(city: str) -> str:
    """Get the weather forecast for a city."""
    return f"Rainy later this week in {city}"

model_with_tools = ChatOpenAI().bind_tools([get_weather, get_forecast])

counts = {"get_weather": 0, "get_forecast": 0, "none": 0}

for i in range(5):
    response = model_with_tools.invoke("what's the weather like")
    if not response.tool_calls:
        counts["none"] += 1
    else:
        picked = response.tool_calls[0]["name"]
        counts[picked] = counts.get(picked, 0) + 1

print(counts)
```

Expected output (an example split — yours may vary run to run):

```
{'get_weather': 3, 'get_forecast': 2, 'none': 0}
```

This version works and gives you a real split. It doesn't print per-run detail, which makes it harder to spot patterns (like "it always picked X on the exact same wording") — worth adding once you're looking closely.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — a reusable trial function with per-run logging

```
from collections import Counter
from langchain_core.tools import tool
from langchain_openai import ChatOpenAI

@tool
def get_weather(city: str) -> str:
    """Get the current weather for a city."""
    return f"Sunny in {city}"

@tool
def get_forecast(city: str) -> str:
    """Get the weather forecast for a city."""
    return f"Rainy later this week in {city}"

def get_model_with_tools() -> ChatOpenAI:
    return ChatOpenAI(model="gpt-4o-mini", temperature=0).bind_tools([get_weather, get_forecast])

def run_selection_trial(prompt: str, runs: int = 5) -> Counter:
    model_with_tools = get_model_with_tools()
    tally: Counter = Counter()

    for run_number in range(1, runs + 1):
        response = model_with_tools.invoke(prompt)
        picked = response.tool_calls[0]["name"] if response.tool_calls else "none"
        print(f"Run {run_number}: picked {picked}")
        tally[picked] += 1

    return tally

if __name__ == "__main__":
```

```
result = run_selection_trial("what's the weather like")
print(f"\nFinal split: {dict(result)}")
```

Expected output (example):

```
Run 1: picked get_weather
Run 2: picked get_forecast
Run 3: picked get_weather
Run 4: picked get_weather
Run 5: picked get_forecast
```

```
Final split: {'get_weather': 3, 'get_forecast': 2}
```

Difference from Basic: printing each run's pick as it happens, not just the final tally, lets you actually look for a pattern (does the exact same prompt always pick the same tool, or does it genuinely vary?) — a detail the Basic version throws away. Using `Counter` instead of a hand-built dictionary also avoids needing a `.get(picked, 0)` fallback, since `Counter` handles unseen keys automatically. Neither version yet checks whether the split itself is trustworthy, or whether it depends on something other than the descriptions — that's what Advanced adds.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Approach 1 — checking whether registration order biases the split

```
from collections import Counter

from langchain_core.tools import BaseTool, tool
from langchain_openai import ChatOpenAI

@tool
def get_weather(city: str) -> str:
    """Get the current weather for a city."""
    return f"Sunny in {city}"

@tool
def get_forecast(city: str) -> str:
    """Get the weather forecast for a city."""
    return f"Rainy later this week in {city}"

def get_model_with_tools(tool_order: list[BaseTool]) -> ChatOpenAI:
    return ChatOpenAI(model="gpt-4o-mini", temperature=0).bind_tools(tool_order)

def run_selection_trial(prompt: str, tool_order: list[BaseTool], runs: int = 5) -> Counter:
    model_with_tools = get_model_with_tools(tool_order)
    tally: Counter = Counter()

    for _ in range(runs):
        response = model_with_tools.invoke(prompt)
        picked = response.tool_calls[0]["name"] if response.tool_calls else "none"
        tally[picked] += 1

    return tally

def main() -> None:
    prompt = "what's the weather like"
```

```

weather_first = run_selection_trial(prompt, [get_weather, get_forecast])
forecast_first = run_selection_trial(prompt, [get_forecast, get_weather])

print(f"Registered [weather, forecast] first: {dict(weather_first)}")
print(f"Registered [forecast, weather] first: {dict(forecast_first)}")

if __name__ == "__main__":
    main()

```

Expected output (example — the real point is comparing the two lines, not the exact numbers):

```

Registered [weather, forecast] first: {'get_weather': 3, 'get_forecast': 2}
Registered [forecast, weather] first: {'get_weather': 3, 'get_forecast': 2}

```

If both lines land close together, order isn't the driver — the wording is, which is what this exercise set out to observe. If the two splits move noticeably (e.g. $4/1$ vs $1/4$), that's a real, separate finding: something about registration order itself is influencing the pick, independent of the description text.

Approach 2 — a bigger sample, to tell signal from noise

```

def run_selection_trial_large(prompt: str, tool_order: list[BaseTool], runs: int
    model_with_tools = get_model_with_tools(tool_order)
    tally: Counter = Counter()

    for _ in range(runs):
        response = model_with_tools.invoke(prompt)
        picked = response.tool_calls[0]["name"] if response.tool_calls else "none"
        tally[picked] += 1

    return tally

result = run_selection_trial_large("what's the weather like", [get_weather, get_f
total = sum(result.values())
for name, count in result.items():
    print(f"{name}: {count}/{total} ({count / total:.0%})")

```

Expected output (example):

```

get_weather: 29/50 (58%)

```

```
get_forecast: 21/50 (42%)
```

A 3/2 split from 5 runs and a 29/21 split from 50 runs both round to "roughly 58/42," but only the 50-run version gives you enough data to actually trust that number instead of it being one lucky or unlucky run away from looking completely different.

Difference from Intermediate, and between these 2 Advanced approaches: Intermediate produces one trustworthy-*looking* split and stops there. Approach 1 checks a hidden variable the single trial never controls for — whether the order tools are registered in shifts the outcome, separate from the wording itself. Approach 2 checks a different hidden variable — whether 5 runs is even enough data to draw a conclusion from at all, versus just noise that happens to look like a pattern.

Which one should you actually write? For a quick, one-off look at whether two descriptions overlap, Intermediate's `run_selection_trial()` at 5 runs is enough — that's genuinely what most people should run first. Reach for Approach 1's order-swap check once you're about to write down a real conclusion ("tool A wins this split") that other people will act on — it's cheap insurance against blaming the wrong variable. Reach for Approach 2's larger sample once the split is close (40/60 or tighter) and the difference actually matters for a decision, since a close split from 5 runs isn't reliable enough to act on either way.